Bit Manipulation

SHSID CPC

Boolean Operations

Input: two (or 1) boolean values (True/False)

Output: a boolean value

	AND	**OR**	**NOT**	**XOR**
Python	a and b	a or b	not a	a != b
C++	a && b	a || b	!a	a != b
Function	True iff both true	True iff at least one true	Returns opposite value	True iff values different

Note: in C++, writing boolean in the “Python way” (i.e. writing a and b, etc.) works, but it is not recommended due to bad compatibility.

Binary Representation

Converting a number to binary should be the easiest thing ever

e.g. 3 = 11, 9 = 1001, 124 = 1111100...

In python there is the function bin(x), but remember that it adds a 0b at the front

In C++... write your own function

Bitwise Operations

Just boolean operations done on every single bit of the number(s)

Input: two (or 1) integers
Output: an integer

The most most common ones are:

	AND	**OR**	**NOT**	**XOR**
Both C++ and Python	a & b	a | b	~a	a ^ b

Note: the NOT operator also acts on the sign of the number if you are using signed integers! Thus, for signed integers, NOT x is equivalent to -x - 1.

Bitwise Operations

Two other very important bitwise operators: Left Shift (<<) and Right Shift (>>).

Effect: a << b = a * (2 ** b) and a >> b = a // (2 ** b)

However, since they are on a more basic level than arithmetic operators, all bitwise operators are much faster than their arithmetic counterparts (e.g. for Python, left shift is 5 to 10 times faster than its corresponding arithmetic operation (of multiplying by a power of 2), while in C++, it is 10 to 100 times faster (!)

Applications

1. lowbit: x & (-x) returns the lowest bit (that is a 1) of x. Used in Binary Index Trees.
2. x & (1 << j) returns the value of x’s 2 ** j bit (i.e. the j+1'th bit from the right). Note that it will return 2 ** j, not 1, if x’s 2 ** j bit exists.
3. x & ~(1 << j) sets x’s j+1'th bit (again, from the right) to 0.
4. x | (1 << j) set x’s j+1'th bit to 1.
5. x ^ (1 << j) inverts x’s j+1'th bit.

.....

Additional Functions

The bitwise functions are not so simple or useful as the common bitwise operators (though they are still used occasionally).

The g++ compiler provides the following functions for counting bits:

- `__builtin_clz`(x): the number of zeros at the beginning of the number
- `__builtin_ctz`(x): the number of zeros at the end of the number
- `__builtin_popcount`(x): the number of ones in the number
- `__builtin_parity`(x): the parity (even or odd) of the number of ones

The functions can be used as follows:

```
int x = 5328; // 00000000000000000001010011010000
cout << __builtin_clz(x) << "\n"; // 19
cout << __builtin_ctz(x) << "\n"; // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n"; // 1
```

While the above functions only support `int` numbers, there are also `long long` versions of the functions available with the suffix `ll`.

Bitset!

A fast way to repeatedly do bitwise operations is to use std::bitset, which gives a 32x or 64x boost to bitwise computation speed when more than 32 or 64 bits are involved (they are slower otherwise), and are used relatively frequently in harder problems.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a = 114514;
    auto start = chrono::high_resolution_clock::now();
    for(int i = 0; i < 100000000; i++){
        __builtin_popcount(a);
    }
    auto end = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> elapsed = end - start;
    cout << "Time taken by normal bitwise: " << elapsed.count() << " ms" << endl;
    bitset<170000> A;
    for (int i = 0; i + 17 <= 170000; i += 17) {
        for (int j = 0; j < 17; j++) {
            A[i + j] = (114514 >> j) & 1;
        }
    }
    start = chrono::high_resolution_clock::now();
    for(int i = 0; i < 10000; i++){
        A.count();
    }
    end = chrono::high_resolution_clock::now();
    elapsed = end - start;
    cout << "Time taken by bitset: " << elapsed.count() << " ms" << endl;
    return 0;
}
```

```
Run: untitled190
/Users/andrew/CLionProjects/untitled190/cmake-build-debug/untitled190
Time taken by normal bitwise: 212.979 ms
Time taken by bitset: 20.069 ms

Process finished with exit code 0
```

A bad example

Sets stored in a number

Suppose you have two sets of many, many items (but assume a small universal set of around a million elements). How should you store them so that you could still perform set operations (like union, difference, intersection, etc.)?

Sets stored in a number

You store an integer (or bitset), where the ith bit stores whether the ith element is in the set or not. Upon some thought, we can see that we could perform set operations on them.

	set syntax	bit syntax
intersection	$a \cap b$	$a \ \& \ b$
union	$a \cup b$	$a \mid b$
complement	$\bar{a}$	$\sim a$
difference	$a \setminus b$	$a \ \& \ (\sim b)$

Longest Common Subsequence

$$\mathbf{dp}[i][j] = \begin{cases} \max(\mathbf{dp}[i-1][j], \mathbf{dp}[i][j-1]) & \text{if } A_i \neq B_j \\ \mathbf{dp}[i-1][j-1] + 1, & \text{if } A_i = B_j \end{cases}$$

	x	***a***	***b***	***c***	***d***
y	**0**	0	0	0	0
a	0	**1**	1	1	1
z	0	**1**	**1**	1	1
c	0	1	1	**2**	**2**

```
class Solution {
  public:
    int longestCommonSubsequence(string a, string b) {
        int dp[a.size()][b.size()];
        for (int i = 0; i < a.size(); i++) { fill(dp[i], dp[i] + b.size(), 0); }
        for (int i = 0; i < a.size(); i++) {
            if (a[i] == b[0]) dp[i][0] = 1;
            if (i != 0) dp[i][0] = max(dp[i][0], dp[i - 1][0]);
        }
        for (int i = 0; i < b.size(); i++) {
            if (a[0] == b[i]) dp[0][i] = 1;
            if (i != 0) dp[0][i] = max(dp[0][i], dp[0][i - 1]);
        }
        for (int i = 1; i < a.size(); i++) {
            for (int j = 1; j < b.size(); j++) {
                if (a[i] == b[j]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[a.size() - 1][b.size() - 1];
    }
};
```

Edit Distance

Edit Distance

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

The *edit distance* between two strings is the minimum number of operations required to transform one string into the other.

The allowed operations are:

- Add one character to the string.
- Remove one character from the string.
- Replace one character in the string.

For example, the edit distance between LOVE and MOVIE is 2, because you can first replace L with M, and then add I.

Your task is to calculate the edit distance between two strings.

Input

The first input line has a string that contains n characters between A–Z.

The second input line has a string that contains m characters between A–Z.

Output

Print one integer: the edit distance between the strings.

Constraints

- $1 \leq n, m \leq 5000$

Example

Input:
```
LOVE
MOVIE
```

Output:
```
2
```

Edit Distance

Edit Distance

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

The *edit distance* between two strings is the minimum number of operations required to transform one string into the other.

The allowed operations are:

- Add one character to the string.
- Remove one character from the string.
- Replace one character in the string.

For example, the edit distance between LOVE and MOVIE is 2, because you can first replace L with M, and then add I.

Your task is to calculate the edit distance between two strings.

Input

The first input line has a string that contains n characters between A–Z.

The second input line has a string that contains m characters between A–Z.

Output

Print one integer: the edit distance between the strings.

Constraints

- $1 \leq n, m \leq 5000$

Example

Input:
```
LOVE
MOVIE
```

Output:
```
2
```

Let DP[i][j] denote th minimum distance to get the first i characters of the first word to the first j characters of the second.

DP[i][j] = ?

MIN:

if (Xi == Yj) DP[i-1][j-1]

else DP[i-1][j-1]+1

DP[i-1][j]

DP[i][j-1]

```java
import java.util.*;

public class EditString {
    public static void main(String[] args){
        Scanner reader = new Scanner(System.in);
        String str1 = ' ' + reader.next();
        String str2 = ' ' + reader.next();
        reader.close();
        int[][] dp = new int[str1.length()+1][str2.length()+1];
        for (int i = 0; i < dp.length; i++){
            Arrays.fill(dp[i], Integer.MAX_VALUE);
        }
        dp[0][0] = 0;
        for (int i = 0; i < str1.length(); i++){
            for (int j = 0; j < str2.length(); j++){
                if (i != 0)
                dp[i][j] = Math.min(dp[i][j], dp[i-1][j]+1);
                if (j != 0)
                dp[i][j] = Math.min(dp[i][j], dp[i][j-1]+1);
                if (i != 0 && j != 0)
                if (str1.charAt(i)==str2.charAt(j))
                dp[i][j] = Math.min(dp[i-1][j-1], dp[i][j]);
                else dp[i][j] = Math.min(dp[i-1][j-1]+1, dp[i][j]);
                // System.out.print(dp[i][j]);
            }
            // System.out.println();
        }
        System.out.println(dp[str1.length()-1][str2.length()-1]);
    }
}
```

Why Cross II

USACO 2017 February Contest, Gold
Problem 2. Why Did the Cow Cross the Road II

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

Farmer John raises N breeds of cows ($1 \le N \le 1000$), conveniently numbered $1 \ldots N$. Some pairs of breeds are friendlier than others, a property that turns out to be easily characterized in terms of breed ID: breeds a and b are friendly if $|a - b| \le 4$, and unfriendly otherwise.

A long road runs through FJ's farm. There is a sequence of N fields on one side of the road (one designated for each breed), and a sequence of N fields on the other side of the road (also one for each breed). To help his cows cross the road safely, FJ wants to draw crosswalks over the road. Each crosswalk should connect a field on one side of the road to a field on the other side where the two fields have friendly breed IDs (it is fine for the cows to wander into fields for other breeds, as long as they are friendly). Each field can be accessible via at most one crosswalk (so crosswalks don't meet at their endpoints).

Given the ordering of N fields on both sides of the road through FJ's farm, please help FJ determine the maximum number of crosswalks he can draw over his road, such that no two intersect.

INPUT FORMAT (file nocross.in):

The first line of input contains N. The next N lines describe the order, by breed ID, of fields on one side of the road; each breed ID is an integer in the range $1 \ldots N$. The last N lines describe the order, by breed ID, of the fields on the other side of the road. Each breed ID appears exactly once in each ordering.

OUTPUT FORMAT (file nocross.out):

Please output the maximum number of disjoint "friendly crosswalks" Farmer John can draw across the road.

SAMPLE INPUT:

```
6
1
2
3
4
5
6
6
5
4
3
2
1
```

SAMPLE OUTPUT:

```
5
```

Problem credits: Brian Dean

Why Cross II

Let DP[i][j] denote th minimum distance to connect the first i on the left and first j on the right

DP[i][j] = min(DP[i-1][j], DP[i][j-1], if friendly(li, rj) DP[i-1][j-1]+1: DP[i-1][j-1]).

Minimum Grid Path

Minimal Grid Path

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

You are given an $n \times n$ grid whose each square contains a letter.

You should move from the upper-left square to the lower-right square. You can only move right or down.

What is the lexicographically minimal string you can construct?

Input

The first line has an integer n: the size of the grid.

After this, there are n lines that describe the grid. Each line has n letters between A and Z.

Output

Print the lexicographically minimal string.

Constraints

- $1 \leq n \leq 3000$

Example

Input:
```
4
AACA
BABC
ABDA
AACA
```

Output:
```
AAABACA
```

Minimum Grid Path

Let DP[i][j] denote the lexicographically minimum string to reach (i, j).

Then, DP[i][j] = min(DP[i-1][j], DP[i][j-1]) + c[i][j].

But that’s up to 3000*3000*6000 comparisons

Minimum Grid Path

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
char c[3005][3005];
bool chosen[3005][3005];
char seq[6005];
int main(){
    cin >> n;
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= n; j++){
            cin >> c[i][j];
        }
    }
    chosen[1][1] = 1;
    for (int i = 3; i <= 2*n; i++){
        char best = 'Z';
        for (int x = 1; x <= min(i, n); x++){
            int y = i-x;
            if (y > n) continue;
            if (chosen[x-1][y] || chosen[x][y-1]){
                if (c[x][y] < best) best = c[x][y];
            }
        }
        for (int x = 1; x <= min(i, n); x++){
            int y = i-x;
            if (y > n) continue;
            if (chosen[x-1][y] || chosen[x][y-1]){
                if (c[x][y] == best) chosen[x][y] = 1;
            }
        }
    }
    int x = n, y = n;
    for (int idx = 2*n-1; idx >= 1; idx--){
        seq[idx] = c[x][y];
        if (chosen[x-1][y]) x--;
        else y--;
    }
    for (int i = 1; i <= 2*n-1; i++){
        cout << seq[i];
    }
    cout << endl;
    return 0;
}
```

E - Swap. AtCoder

Problem Statement

Given is a string S consisting of K, E, Y.

How many strings are there that can be obtained with at most K swaps of two adjacent characters in S?

Constraints

- $2 \leq |S| \leq 30$
- $0 \leq K \leq 10^9$
- S consists of K, E, Y.

E - Swap. AtCoder

Firstly, the k is massive and scary. But notice how we can achieve every possible configuration within 30*29/2 = 435 swaps, by a bubble sort sorta idea.

Then, for a fixed final configuration, the way to optimize for the number of swaps is obvious: complete from left to right.

Let DP[cost][k][e][y] denote the number of ways to complete the first k+e+y with k Ks, e E's and y Y's, with cost cost.

Then, DP[cost + abs(k'th K position, k+e+y)][k][e][y]